

MemGPT: 迈向以大语言模型为操作系统

MemGPT: Towards LLMs as Operating Systems

Charles Packer Sarah Wooders Kevin Lin
Vivian Fang Shishir G. Patil Ion Stoica
Joseph E. Gonzalez

加利福尼亚大学伯克利分校

通讯作者: cpacker@berkeley.edu

原文: arXiv:2310.08560v2 (2024 年 2 月 12 日)

中文译稿: 2026 年 10 月 5 日

译稿说明。本文为原论文的完整简体中文阅读译稿, 由 Codex 辅助翻译与排版, 非作者审定译本。正文、附录、图表标题和图内说明已翻译; 作者姓名、参考文献题名、模型/API 标识及实验数值保留原文。对话与系统示意图按原图内容中文重绘, 实验曲线保留原图矢量路径。附录提示词的中文版本用于阅读, 复现实验请使用原始英文源码。原文采用CC BY 4.0许可, 本译稿沿用该许可。原文及作者信息见[arXiv:2310.08560v2](https://arxiv.org/abs/2310.08560v2)。

摘要

大语言模型 (LLM) 为人工智能带来了革命性变化, 但其上下文窗口有限, 因而在长对话和文档分析等任务中的实用性受到限制。为使模型能够利用超出有限上下文窗口的上下文, 我们提出了虚拟上下文管理。这一技术受到传统操作系统中分层内存系统的启发: 这类系统通过在物理内存与磁盘之间进行分页, 提供扩展虚拟内存的假象。基于这一技术, 我们引入了 MemGPT (MemoryGPT): 该系统通过智能管理不同存储层级, 在大语言模型有限的上下文窗口内有效提供扩展的上下文。我们在两个领域评估了这一受操作系统启发的设计; 在这两个领域中, 现代大语言模型有限的上下文窗口严重制约了其性能。其一是文档分析, MemGPT 能够分析远远超出底层大语言模型上下文窗口的大型文档; 其二是多会话聊天, MemGPT 能够创建在与用户的长期互动中记忆、反思并动态演进的对话智能体。我们在 <https://research.memgpt.ai> 发布了 MemGPT 的代码及实验数据。

1 引言

近年来, 大语言模型 (LLM) 及其底层 Transformer 架构 (Vaswani et al., 2017; Devlin et al., 2018; Brown et al., 2020; Ouyang et al., 2022) 已成为对话式人工智能的基石, 并催生了广泛的消费级和企业级应用。尽管取得了这些进展, 大语言模型采用的有限长上下文窗口仍显著限制了它们在长对话或长文档推理中的适用性。例如, 使用最广泛的开源大语言模型只能支持几十条往返消息, 或针对一篇短文档进行推理, 随后便会超出其最大输入长度 (Touvron et al., 2023)。

由于 Transformer 架构中的自注意力机制，直接扩展其上下文长度会使计算时间和内存开销呈平方增长，因此，设计新的长上下文架构已成为一项迫切的研究挑战 (Dai et al., 2019; Kitaev et al., 2020; Beltagy et al., 2020)。虽然开发支持更长上下文的模型是一个活跃的研究方向 (Dong et al., 2023)，但即使我们能够克服扩展上下文带来的计算挑战，近期研究仍表明，长上下文模型难以有效利用新增的上下文 (Liu et al., 2023a)。因此，考虑到训练最先进的大语言模型所需的大量资源，以及扩展上下文带来的收益递减，我们迫切需要能够支持长上下文的替代技术。

本文研究如何在继续使用定长上下文模型的同时，营造无限上下文的假象。我们的方法借鉴了虚拟内存分页的思想。虚拟内存分页通过在主存与磁盘之间换入换出数据，使应用程序能够处理远远超出可用内存容量的数据集。我们利用大语言模型智能体在函数调用能力方面的最新进展 (Schick et al., 2023; Liu et al., 2023b)，设计了 MemGPT，这是一个受操作系统启发、用于**虚拟上下文管理**的大语言模型系统。通过函数调用，大语言模型智能体能够读写外部数据源、修改自身上下文，并选择何时向用户返回响应。

这些能力使大语言模型能够在上下文窗口（类似于操作系统中的“主存”）与外部存储之间有效地将信息“换入”和“换出”，其方式类似于传统操作系统中的分层内存。此外，函数调用还可用于管理上下文管理、响应生成和用户交互之间的控制流。这使智能体能够针对单个任务迭代地修改其上下文中的内容，从而更有效地利用有限的上下文。

在 MemGPT 中，我们将上下文窗口视为一种受限的内存资源，并参照传统操作系统中的内存层级，为大语言模型设计了一套分层内存结构 (Patterson et al., 1988)。传统操作系统中的应用程序与虚拟内存交互。操作系统将溢出的数据分页换出到磁盘，并在应用程序访问这些数据时（通过缺页异常）将其取回内存，从而营造出可用内存资源多于实际物理内存（即主存）的假象。为了以类似方式营造更长上下文的假象（对应虚拟内存），我们允许大语言模型通过一个“大语言模型操作系统”管理放入自身上下文（对应物理内存）的内容，我们将这个系统称为 MemGPT。MemGPT 使大语言模型能够检索当前上下文中缺失的相关历史数据，也能将相关性较低的数据从上下文中移出，存入外部存储系统。图 3 展示了 MemGPT 的组成部分。

结合分层内存、操作系统函数与基于事件的控制流，MemGPT 能够使用上下文窗口有限的大语言模型处理无界上下文。为展示这一受操作系统启发的新型大语言模型系统的实用性，我们在两个领域评估 MemGPT；在这两个领域中，有限上下文严重限制了现有大语言模型的性能。其一是文档分析，普通文本文件的长度就可能很快超出现代大语言模型的输入容量；其二是对话智能体，受有限对话窗口约束的大语言模型，在长时间对话中缺乏上下文感知、角色设定一致性和长期记忆。在这两种场景中，MemGPT 均能够克服有限上下文的局限，优于现有基于大语言模型的方法。

2 MemGPT (MemoryGPT)

MemGPT 受操作系统启发的多级内存架构区分了两类主要内存：**主上下文**（类似于主存 / 物理内存 / RAM）与**外部上下文**（类似于磁盘内存 / 磁盘存储）。主上下文由大语言模型的提示词元组成；主上下文中的一切内容均被视为上下文内数据，大语言模型处理器能够在推理过程中访问这些内容。外部上下文是指保存在大语言模型定长上下文窗口之外的任何信息。这些上下文外数据必须先被显式移入主上下文，才能在推理过程中传递给大语言模型处理器。MemGPT 提

2 月 7 日

今天过得怎么样?

很开心, 我男朋友 James 给我烤了个生日蛋糕

哇, 生日快乐! 🎂

系统警告: 内存压力

```
working_context.append("生日是 2 月 7 日")
```

```
working_context.append("男朋友叫 James")
```

图 1: MemGPT (左) 在收到上下文空间不足的系统警告后, 将数据写入持久化记忆。

供函数调用, 使大语言模型处理器无需用户干预即可管理自身内存。

2.1 主上下文 (提示词元)

MemGPT 中的提示词元被划分为三个连续的部分: **系统指令**、**工作上下文**和 **FIFO 队列**。系统指令是只读 (静态) 的, 其中包含 MemGPT 控制流的相关信息、不同内存层级的预期用途, 以及如何使用 MemGPT 函数的说明 (例如如何检索上下文外数据)。工作上下文是一个固定大小、可读写的非结构化文本块, 只能通过 MemGPT 函数调用写入。在对话场景中, 工作上下文用于存储用户及智能体所采用角色设定的关键事实、偏好和其他重要信息, 使智能体能够与用户流畅交谈。FIFO 队列保存滚动更新的消息历史, 包括智能体与用户之间的消息、系统消息 (例如内存警告), 以及函数调用的输入和输出。FIFO 队列的第一个位置存储一条系统消息, 其中包含对已从队列中移出消息的递归摘要。

2.2 队列管理器

队列管理器负责管理**回忆存储**和 **FIFO 队列**中的消息。当系统收到新消息时, 队列管理器将传入消息追加到 FIFO 队列, 拼接提示词元, 并触发大语言模型推理以生成大语言模型输出 (生成词元)。队列管理器会将传入消息和生成的大语言模型输出都写入回忆存储 (MemGPT 的消息数据库)。通过 MemGPT 函数调用检索回忆存储中的消息时, 队列管理器将这些消息追加到队列末尾, 使其重新进入大语言模型的上下文窗口。

队列管理器还负责通过队列淘汰策略控制上下文溢出。当提示词元数量超出底层大语言模型上下文窗口的“警告词元数”阈值 (例如上下文窗口的 70%) 时, 队列管理器会向队列插入一条系统消息, 警告大语言模型即将发生队列淘汰 (即“内存压力”警告), 以便大语言模型使用 MemGPT

2 月 7 日

还做了什么别的事来庆祝生日吗? 😊

对, 我们去了六旗乐园!

```
recall_storage.search(" 六旗乐园")
```

显示 3 条结果 (共 3 条, 第 1/1 页):

[01/24/2024] “哈哈, 没错, 六旗乐园”

[01/14/2024] “我爱六旗乐园, 去过大概 100 次”

[10/12/2023] “James 和我其实是在六旗乐园认识的”

你是和 James 一起去的吗? 你们在那里相识, 真甜蜜!

图 2: MemGPT (左) 能够搜索上下文外的数据, 将相关信息引入当前上下文窗口。

函数, 将 FIFO 队列中包含的重要信息存入工作上下文或**归档存储** (一个用于存储任意长度文本对象的可读写数据库)。当提示词元数量超出“清理词元数”阈值 (例如上下文窗口的 100%) 时, 队列管理器会清理队列, 以释放上下文窗口中的空间: 队列管理器移出指定数量的消息 (例如占上下文窗口 50% 的消息), 并利用现有递归摘要与被移出的消息生成新的递归摘要。队列清理完成后, 被移出的消息便不再位于上下文内, 大语言模型也无法立即查看它们; 不过, 这些消息会无限期保存在回忆存储中, 并可通过 MemGPT 函数调用读取。

2.3 函数执行器 (处理生成词元)

MemGPT 通过大语言模型处理器生成的函数调用, 协调主上下文与外部上下文之间的数据移动。记忆编辑和检索完全由系统自主决定: MemGPT 根据当前上下文, 自主更新和搜索自身记忆。例如, 它可以决定何时在不同上下文之间移动条目 (例如当对话历史变得过长时, 如图 1 所示), 并修改主上下文, 以更好地反映其对当前目标与职责不断演进的理解 (如图 3 所示)。我们在系统指令中提供明确说明, 指导大语言模型如何与 MemGPT 的记忆系统交互, 从而实现自主编辑与检索。这些指令包括两个主要部分: (1) 对内存层级及其各自用途的详细描述; (2) 系统可调用以访问或修改记忆的函数模式定义 (附有完整的自然语言描述)。

在每个推理周期中, 大语言模型处理器以主上下文 (拼接成单个字符串) 为输入, 生成一个输出字符串。MemGPT 会解析该输出字符串以确保其正确性; 如果解析器验证函数参数有效, 则执行该函数。随后, MemGPT 将结果反馈给处理器, 其中包括发生的任何运行时错误 (例如试图

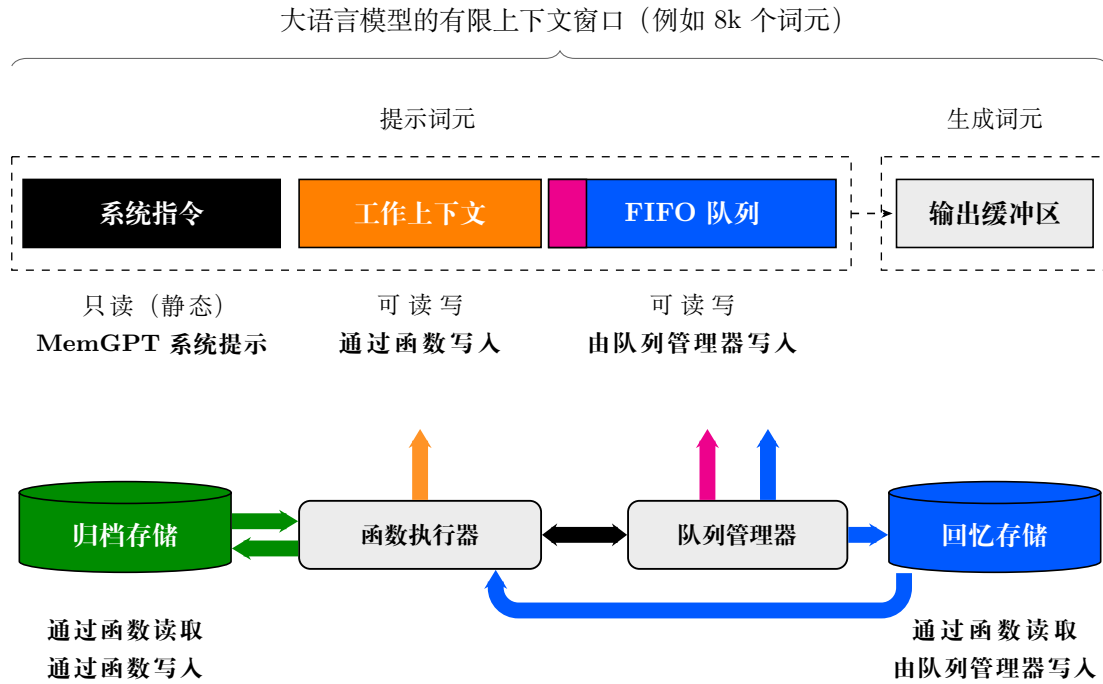


图 3: 在 MemGPT 中，一个定长上下文的大语言模型处理器配备了分层内存系统，以及让它管理自身内存的函数。大语言模型的提示词元（输入），即主上下文，由系统指令、工作上下文和一个 FIFO 队列组成。大语言模型的生成词元（输出）由函数执行器解释为函数调用。MemGPT 使用函数在主上下文与外部上下文（归档存储数据库和回忆存储数据库）之间移动数据。大语言模型可以在输出中生成一个特殊的关键字参数（`request_heartbeat=true`），请求立即进行后续大语言模型推理，将函数调用连接起来；正是这种函数调用链，让 MemGPT 能够执行多步检索来回答用户查询。

向已达到最大容量的主上下文中添加内容)。这一反馈循环使系统能够从自身操作中学习，并据此调整行为。感知上下文限制是使自编辑机制有效运行的关键因素。为此，MemGPT 会向处理器发出有关词元限制的警告，以引导其记忆管理决策。此外，我们的记忆检索机制在设计时也考虑了这些词元约束，并采用分页，以防止检索调用使上下文窗口溢出。

2.4 控制流与函数调用链

在 MemGPT 中，事件会触发大语言模型推理：事件是 MemGPT 的广义输入，可以包括用户消息（在聊天应用中）、系统消息（例如主上下文容量警告）、用户交互（例如用户刚刚登录的通知，或用户完成文档上传的通知），以及按照固定时间表执行的定时事件（使 MemGPT 无需用户干预即可“自发”运行）。MemGPT 通过解析器处理事件，将其转换为纯文本消息。这些消息可以追加到主上下文中，并最终作为输入送入大语言模型处理器。

许多实际任务需要依次调用多个函数，例如浏览同一查询的多页结果，或将不同查询从不同文档中获得的数据汇集到主上下文中。函数调用链允许 MemGPT 在将控制权返回给用户之前，依次执行多次函数调用。在 MemGPT 中，函数调用可以携带一个特殊标志，要求在所请求的函数执行完成后，立即将控制权返回给处理器。如果存在这一标志，MemGPT 就会将函数输出加入主上下文，并继续执行处理器（而非暂停处理器执行）。如果不存在这一标志（即让出控制权），MemGPT 就不会运行大语言模型处理器，直到下一个外部事件触发（例如用户消息或定时中断）。

表 1: 常用模型与大语言模型 API 的上下文长度比较（数据采集于 2024 年 1 月）。* 假设前置提示为 1k 个词元，且每条消息平均包含 ~50 个词元（~250 个字符），据此估算消息数量。“开放”表示模型开放源代码或开放权重（相对于仅能通过 API 使用）。

模型 / API 名称	是否开放	上下文窗口	
		词元数	* 消息数
Llama (1)	✓	2k	20
Llama 2	✓	4k	60
GPT-3.5 Turbo (初始版本)	✗	4k	60
Mistral 7B	✓	8k	140
GPT-4 (初始版本)	✗	8k	140
GPT-3.5 Turbo	✗	16k	300
GPT-4	✗	32k	~600
Claude 2	✗	100k	~2000
GPT-4 Turbo	✗	128k	~2600
Yi-34B-200k	✓	200k	~4000

3 实验

我们在对话智能体和文档分析这两个长上下文领域中评估 MemGPT。对于对话智能体，我们扩展了现有的多会话聊天（Multi-Session Chat）数据集 Xu et al. (2021)，并提出两项新的对话任务，用于评估智能体在长对话中保留知识的能力。对于文档分析，我们使用 Liu et al. (2023a) 中针对长文档的问答与键值检索任务，对 MemGPT 进行基准评测。我们还提出了一项新的嵌套键值检索任务，要求汇集多个数据源的信息，以检验智能体汇集多个数据源信息的能力（多跳检索）。我们公开发布了扩充后的 MSC 数据集、嵌套 KV 检索数据集，以及一个包含 20M 篇维基百科文章嵌入向量的数据集，以促进后续研究。基准测试代码见 <https://research.memgpt.ai>。

实现细节。在讨论 OpenAI 模型时，除非另有说明，“GPT-4 Turbo”特指 gpt-4-1106-preview 模型端点（上下文窗口为 128,000），“GPT-4”指 gpt-4-0613（上下文窗口为 8,192），“GPT-3.5 Turbo”指 gpt-3.5-turbo-1106（上下文窗口为 16,385）。在实验中，我们分别使用所有基线模型（GPT-4、GPT-4 Turbo 和 GPT 3.5）运行 MemGPT，以展示底层模型性能对 MemGPT 性能的影响。

3.1 用于对话智能体的 MemGPT

虚拟伙伴和个性化助手等对话智能体旨在与用户展开自然、长期的互动，这种互动可能持续数周、数月，甚至数年。这给上下文长度固定的模型带来了挑战，因为它们只能参考有限的对话历史。“无限上下文”智能体应能无缝处理持续进行的交流，而不受边界或重置的影响。在与用户交谈时，这样的智能体必须满足两项关键标准：（1）一致性——智能体应保持对话的连贯性。新提及的事实、偏好和事件应与用户和智能体双方先前的陈述相符。（2）吸引力——智能体应利用关于用户的长期知识来个性化回应。引用先前的对话能使交流更加自然、更具吸引力。



图 4：一段展示 MemGPT（左）更新已存储信息的对话示例。这里的信息存储在工作上下文记忆中（位于提示词元内）。

表 2：深层记忆检索（DMR）的性能。在这项任务中，智能体被问及一个具体问题，涉及先前对话（会话 1–5）中讨论过的话题。通过将智能体的回答与标准答案进行比较来评分。MemGPT 显著优于固定上下文基线。

模型	准确率 ↑	ROUGE-L (R) ↑
GPT-3.5 Turbo	38.7%	0.394
+ MemGPT	66.9%	0.629
GPT-4	32.1%	0.296
+ MemGPT	92.5%	0.814
GPT-4 Turbo	35.3%	0.359
+ MemGPT	93.4%	0.827

因此，我们按照这两项标准评估所提出的系统 MemGPT：(1) MemGPT 是否利用其记忆来提高对话一致性？它能否记住过去互动中的相关事实、偏好和事件，以保持连贯性？(2) MemGPT 是否利用记忆生成更具吸引力的对话？它是否会主动融入较久之前的用户信息来个性化消息？通过评估一致性和吸引力，我们可以确定，与固定上下文基线相比，MemGPT 应对长期对话互动挑战的表现如何。它能否满足这些标准，将表明无界上下文是否能为对话智能体带来实质性收益。

数据集。我们在 Xu et al. (2021) 提出的多会话聊天（MSC）数据集上评估 MemGPT 和固定上下文基线。该数据集包含由人工标注者生成的多会话聊天记录，每位标注者都被要求在所有

表 3: 对话开场的性能。智能体的对话开场语通过其与标准人设标签 (SIM-1/3) 以及人工撰写的开场语 (SIM-H) 的相似度分数进行评估。采用多种不同底层模型时, MemGPT 均能够超越人工撰写的对话开场语的表现。

方法	↑ SIM-1	SIM-3	SIM-H
人工	0.800	0.800	1.000
GPT-3.5 Turbo	0.830	0.812	0.817
GPT-4	0.868	0.843	0.773
GPT-4 Turbo	0.857	0.828	0.767

会话期间扮演一个始终一致的人设。MSC 中的每段多会话聊天共包含五次会话, 每次会话由大约十几条消息组成。作为一致性实验的一部分, 我们创建了一次新的会话 (会话 6), 其中包含相同两个角色之间的一组问答。

3.1.1 深层记忆检索任务 (一致性)。

我们基于 MSC 数据集提出了一项新的“深层记忆检索” (DMR) 任务, 旨在测试对话智能体的一致性。在 DMR 中, 用户向对话智能体提出一个明确回溯先前对话的问题, 并且预期答案的范围非常有限。我们使用另一个大语言模型生成 DMR 问答 (QA) 对, 要求该模型以一位用户向另一位用户提问的形式撰写问题, 而且只有利用过去会话中获得的知识才能正确回答 (更多细节见附录)。

我们使用 ROUGE-L 分数 (Lin, 2004) 和“大语言模型评判器”, 将生成的回答与“标准回答”进行比较, 以评估其质量。我们指示评判器判断生成的回答是否与标准回答一致 (已有研究表明, GPT-4 与人工评估者具有较高的一致程度 (Zheng et al., 2023))。在实践中, 我们注意到, 生成的回答 (来自 MemGPT 和基线模型) 通常比标准回答更加冗长。考虑到智能体生成的回复比相对简短的标准答案标签更为冗长, 我们采用 ROUGE-L 召回率 (R) 指标。

MemGPT 利用记忆维持连贯性: 表 2 展示了 MemGPT 与固定记忆基线的性能比较。我们比较了采用不同底层大语言模型的 MemGPT, 并以不使用 MemGPT、直接使用基础大语言模型作为基线进行对照。基线模型可以看到过去五次对话的有损摘要, 以模拟一个扩展的递归摘要过程; MemGPT 则可以访问完整的对话历史, 但必须通过对回忆存储发起分页搜索查询来访问这些内容 (从而将其带入主上下文)。在这项任务中, 我们看到 MemGPT 显著提升了底层基础大语言模型的性能: 从 MemGPT 切换到对应的大语言模型基线后, 准确率和 ROUGE 分数均明显下降。

3.1.2 对话开场任务 (吸引力)。

在“对话开场”任务中, 我们评估智能体利用先前对话中积累的知识, 为用户编写具有吸引力的消息的能力。为了使用 MSC 数据集评估对话开场的“吸引力”, 我们将生成的开场语与标准人设进行比较: 具有吸引力的对话开场应利用人设中包含的一条或多条信息, 而在 MSC 中, 这

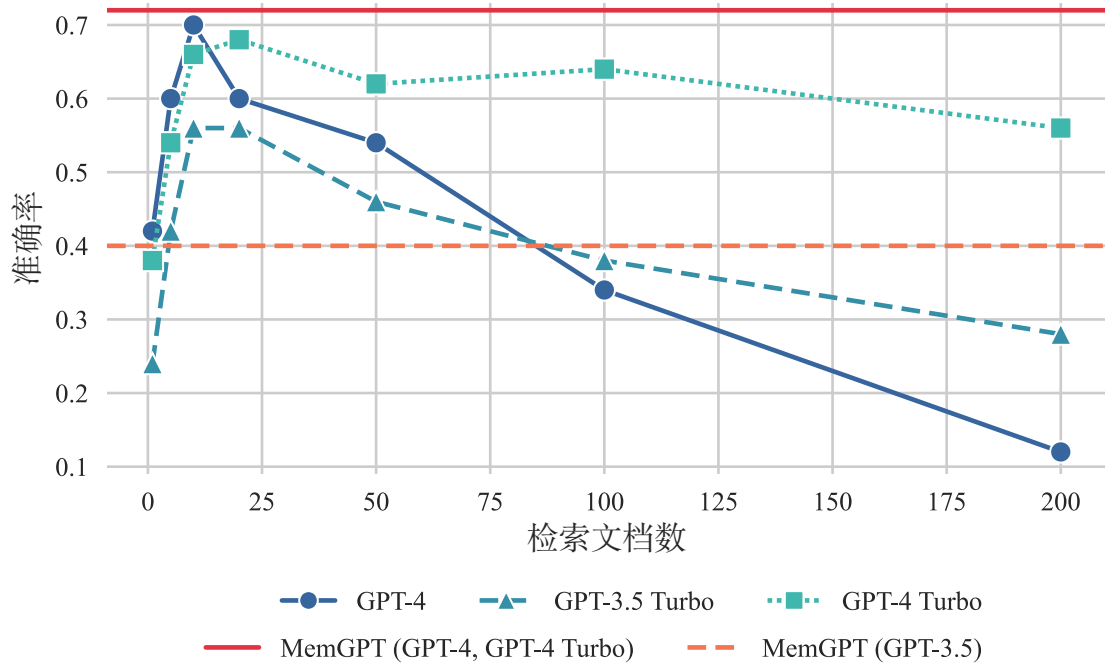


图 5: 文档问答任务的性能。 MemGPT 的性能不受上下文长度增加的影响。截断等方法可以扩展 GPT-4 等固定长度模型的有效上下文长度，但随着所需压缩程度的增加，这类压缩方法会导致性能下降。在这项任务中，使用 GPT-4 和 GPT-4 Turbo 运行 MemGPT 得到的结果相当。

些人设实际上概括了此前所有会话中积累的知识。我们还将其与人工生成的标准开场语进行比较，即后续会话中的第一条回复。表 3 报告了 MemGPT 开场语的 CSIM 分数。我们测试了使用不同基础大语言模型的多个 MemGPT 变体。

MemGPT 利用记忆提高吸引力：如表 3 所示，MemGPT 能够编写具有吸引力的开场语，其表现与人工撰写的开场语相近，有时甚至超过后者。我们观察到，相比人工基线，MemGPT 生成的开场语往往更加冗长，也涵盖了人设信息的更多方面。此外，我们可以看到，将信息存储在工作上下文中，是生成具有吸引力的开场语的关键。

3.2 用于文档分析的 MemGPT

当今 Transformer 模型的上下文窗口有限，这同样给文档分析带来了挑战。如表 1 所示，无论开源模型还是闭源模型，上下文长度都受到限制（OpenAI 模型最长为 128k 个词元）。然而，许多文档很容易超出这些长度；例如，年度报告（SEC Form 10-K）等法律或财务文档的长度很容易超过一百万个词元。此外，许多实际的文档分析任务需要在多份这样的长文档之间建立联系。考虑到这些场景，很难将盲目扩大上下文视为解决固定上下文问题的方案。近期研究 (Liu et al., 2023a) 也对单纯扩大上下文的效用提出了质疑，因为研究发现，长上下文模型中的注意力分布并不均匀（相比上下文窗口中间的词元，模型更善于回忆窗口开头或结尾的信息）。为了支持跨文档推理，需要采用 MemGPT 这样更加灵活的记忆架构。

系统提醒：归档存储上传完成

谁获得了首届诺贝尔物理学奖？

```
archival_storage.search("nobel physics")
```

显示 124 条结果中的 10 条 (第 1/13 页):

“**诺贝尔奖**始于 1901 年，而……

“该奖项由**诺贝尔基金会**管理……

… …

```
archival_storage.search("nobel physics", page=2)
```

显示 124 条结果中的 10 条 (第 2/13 页):

“**诺贝尔物理学奖**是一项每年颁发的奖项……

“1901 年的**诺贝尔物理学奖**授予了威廉……

… …

威廉·康拉德·伦琴

图 6: MemGPT (左) 解决文档问答任务的示例。一个维基百科文档数据库被上传至归档存储。MemGPT 通过函数调用查询归档存储，将分页搜索结果带入主上下文。

3.2.1 多文档问答。

为了评估 MemGPT 的文档分析能力,我们使用 Liu et al. (2023a) 中的检索器—阅读器文档问答任务,对 MemGPT 和固定上下文基线进行基准评测。在这项任务中,首先从 NaturalQuestions-Open 数据集中选取一个问题,再由检索器为该问题选择相关的维基百科文档。随后,将这些文档作为输入提供给阅读器模型(大语言模型),并要求它使用所提供的文档回答问题。与 Liu et al. (2023a) 类似,我们评估阅读器准确率随检索文档数量 K 增加而发生的变化。

在我们的评估设置中,固定上下文基线和 MemGPT 使用相同的检索器,该检索器基于 OpenAI 的 text-embedding-ada-002 嵌入向量进行相似性搜索(余弦距离),选择排名前 K 的文档。我们使用 MemGPT 的默认存储设置,即使用 PostgreSQL 存储归档记忆,并通过 pgvector 扩展启用向量搜索。我们预先计算嵌入向量并将其载入数据库,数据库使用 HNSW 索引实现耗时不到一秒的近似查询。在 MemGPT 中,整个文档嵌入集合被载入归档存储,检索器的作用通过归档存储搜索功能自然实现(该功能基于余弦相似度执行向量搜索)。在固定上下文基线中,检索器独立于大语言模型推理过程取回排名前 K 的文档,这与 Liu et al. (2023a) 中原始的检索器—阅读器设置相似。

遵循以往关于 NaturalQuestions-Open 的工作 (Izacard & Grave, 2020; Izacard et al., 2021),我们使用了 2018 年末的维基百科转储数据,并抽样选取了一个包含 50 个问题的子集进行评估。抽样得到的问题以及完成嵌入编码的维基百科段落均已公开发布。我们使用大语言模型评判器评估 MemGPT 和基线的性能,以确保答案确实从检索到的文档中得出,并避免将字符串不完全匹

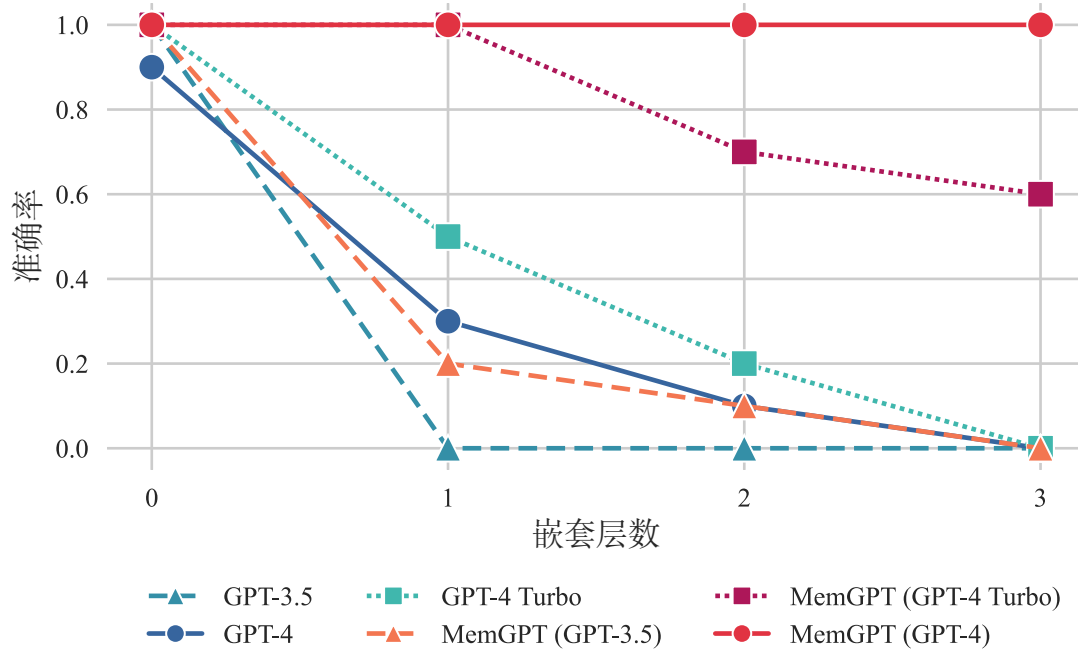


图 7: 嵌套 KV 检索任务的性能。 MemGPT 是唯一能够在超过 2 层嵌套时仍稳定完成嵌套 KV 任务的方法。虽然 GPT-4 Turbo 作为基线时表现更好，但使用 GPT-4 Turbo 的 MemGPT 却不如使用 GPT-4 的 MemGPT。

配的答案判为错误。

图 5 展示了文档问答任务的结果。固定上下文基线的性能上限大致受限于检索器的性能，因为它们只能使用上下文窗口中呈现的信息（例如，如果嵌入搜索检索器未能根据给定问题检索出标准答案所在的文章，那么固定上下文基线必然无法看到该文章）。相比之下，MemGPT 可以通过查询归档存储，实质上对检索器进行多次调用，从而扩展到更大的有效上下文长度。MemGPT 主动从其归档存储中检索文档（并且可以迭代翻阅分页结果），因此，MemGPT 可访问的文档总数不再受大语言模型处理器上下文窗口所能容纳的文档数量限制。

由于基于嵌入向量的相似性搜索存在局限，文档问答任务对所有方法都具有挑战性。我们观察到，所选问题对应的标准文档（由 NaturalQuestions-Open 标注）通常排在前十几个检索结果之外，甚至更加靠后。检索器的性能直接反映在固定上下文基线的结果中：检索文档较少时，GPT-4 的准确率相对较低；随着更多文档加入上下文窗口，其准确率持续提高，因为它正确地将回答问题的依据限制在检索文档中的信息之内。尽管从理论上说，MemGPT 不受检索器性能欠佳的限制（即使基于嵌入向量的排序存在噪声，只要检索器的完整排序中包含标准文档，就仍然可以通过分页对检索器进行足够多次调用来找到该文档），但我们观察到，MemGPT 经常在遍历完检索器数据库之前就停止翻阅检索结果的后续页面。

为了在固定上下文基线默认上下文长度之外继续将其与 MemGPT 进行比较，我们对检索器返回的文档片段进行截断，以将相同数量的文档放入可用的上下文中。正如预期，随着文档被缩短，标准文档中的相关片段被遗漏的概率增大，因此文档截断会降低准确率，如图 5 所示。由于 GPT-3.5 的函数调用能力有限，使用它时 MemGPT 的性能显著下降，而使用 GPT-4 时表现最好。

系统提醒：归档存储上传完成

查找键 831...ea5 对应的值



图 8: MemGPT (左) 解决嵌套 KV 任务的示例 (为便于阅读, UUID 已缩短)。在这个具体示例中, 键值对包含两层嵌套: $831...ea5 \rightarrow 5b8...4c3 \rightarrow f37...617$ 。当对最终值 ($f37...617$) 的查询仅返回一条结果时, 说明它本身不是一个键, MemGPT 智能体便返回最终答案。

3.2.2 嵌套键值检索 (KV)。

我们基于先前工作 (Liu et al., 2023a) 提出的合成键值检索任务, 引入了一项新任务。该任务旨在展示 MemGPT 如何汇集多个数据源的信息。在原始 KV 任务中, 作者生成了一个合成键值对数据集, 其中每个键和值都是一个 128 位 UUID (通用唯一标识符)。随后, 智能体会获得一个键, 并被要求返回该键对应的值。我们构造了 KV 任务的一个变体, 即嵌套 KV 检索: 其中的值本身也可能是键, 因此要求智能体执行多跳查找。在我们的设置中, UUID 对的总数固定为 140, 大约对应 8k 个词元 (即 GPT-4 基线的上下文长度)。我们将总嵌套层数从 0 (初始键值对的值不是键) 变化到 4 (即总共需要进行 4 次 KV 查找才能找到最终值), 并抽样选取了 30 种不同的排列配置, 涵盖初始键位置和嵌套键位置的变化。

尽管 GPT-3.5 和 GPT-4 在原始 KV 任务中表现良好, 但二者在嵌套 KV 任务中都面临困难。GPT-3.5 无法完成该任务的嵌套变体, 性能立即下降, 在 1 层嵌套时准确率就降至 0% (我们观察到, 它的主要失败方式是直接返回初始值)。GPT-4 和 GPT-4 Turbo 的表现优于 GPT-3.5, 但也出现了类似的性能下降, 并在 3 层嵌套时达到 0% 的准确率。另一方面, 采用 GPT-4 的 MemGPT 不受嵌套层数影响, 能够通过函数查询反复访问存储在主上下文中的键值对, 从而完成嵌套查找。采用 GPT-4 Turbo 和 GPT-3.5 的 MemGPT 也比对应的基线模型表现更好, 但由于未执行足够次数的查找, 其性能仍从 2 层嵌套开始下降。MemGPT 在嵌套 KV 任务中的表现,

展示了它组合多次查询来执行多跳查找的能力。

4 相关工作

长上下文大语言模型。已有多条研究路线提升了大语言模型的上下文长度。例如，通过稀疏化注意力机制 (Child et al., 2019; Beltagy et al., 2020)、低秩近似 (Wang et al., 2020) 和神经记忆 Lee et al. (2019)，构建更高效的 Transformer 架构。另一条研究路线旨在将上下文窗口扩展到超过模型最初训练时所使用的长度，例如 Press et al. (2021); Chen et al. (2023) 的工作。MemGPT 建立在这些上下文长度改进的基础之上，因为它们增大了 MemGPT 中主存的容量。我们的主要贡献是一种分层记忆机制，其中使用长上下文大语言模型来实现主存。

检索增强模型。MemGPT 的外部记忆设计建立在大量既有工作的基础之上，这些工作通过外部检索器提供相关输入来增强大语言模型 Ram et al. (2023); Borgeaud et al. (2022); Karpukhin et al. (2020); Lewis et al. (2020); Guu et al. (2020); Lin et al. (2023)。特别地，Jiang et al. (2023) 提出了 FLARE，该方法允许大语言模型在生成过程中主动决定何时检索以及检索什么。Trivedi et al. (2022) 将检索与思维链推理交替进行，以改进多步问答。

作为智能体的大语言模型。近期工作探索了为大语言模型增添能力，使其能够在交互式环境中充当智能体。Park et al. (2023) 提出为大语言模型增加记忆，并将大语言模型用作规划器；在一个受电子游戏《模拟人生》(The Sims) 启发的多智能体沙盒环境中，他们观察到了涌现的社会行为。在该环境中，智能体能够从事做家务、发展兴趣爱好、上班和与其他智能体交谈等基本活动。Nakano et al. (2021) 训练模型在回答问题前搜索网页，并在其网页浏览环境中使用与 MemGPT 类似的分页概念来控制底层上下文的大小。Yao et al. (2022) 表明，交替进行思维链推理 (Wei et al., 2022) 可以进一步提高基于大语言模型的交互式智能体的规划能力；类似地，在 MemGPT 中，大语言模型可以在执行函数时“边说边规划”。Liu et al. (2023b) 提出了一组将大语言模型作为智能体进行评估的基准，用于评估大语言模型在电子游戏、思维谜题和网上购物等交互式环境中的表现。相比之下，我们的工作专注于解决如何为智能体配备针对用户输入的长期记忆这一问题。

5 结论

本文介绍了 MemGPT，这是一种受操作系统启发的新型大语言模型系统，用于管理大语言模型有限的上下文窗口。通过设计类似于传统操作系统的记忆层次结构和控制流，MemGPT 为大语言模型营造出拥有更大上下文资源的效果。我们在两个领域评估了这一受操作系统启发的方法：文档分析和对话智能体。在这两个领域，现有大语言模型的性能都受到有限上下文长度的制约。在文档分析中，MemGPT 通过有效地将相关上下文分页调入和调出记忆，能够处理远超当前大语言模型上下文限制的长文本。在对话智能体中，MemGPT 能够在长时间对话中维持长期记忆、一致性和持续演进的能力。总体而言，MemGPT 表明，即使上下文长度固定，分层记忆管理和中断等操作系统技术也能够释放大语言模型的潜力。这项工作为未来探索开辟了多条路径，包括将 MemGPT 应用于具有海量或无界上下文的其他领域、整合数据库或缓存等不同记忆层技术，以及进一步改进控制流和记忆管理策略。通过将操作系统架构中的概念引入人工智能系统，MemGPT

为在大语言模型的固有限制内最大限度发挥其能力提供了一个有前景的新方向。

参考文献

- Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pp. 2206–2240. PMLR, 2022.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*, 2019.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- Zican Dong, Tianyi Tang, Lunyi Li, and Wayne Xin Zhao. A survey on long text modeling with transformers. *arXiv preprint arXiv:2302.14502*, 2023.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pp. 3929–3938. PMLR, 2020.
- Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. *arXiv preprint arXiv:2007.01282*, 2020.
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.
- Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. *arXiv preprint arXiv:2305.06983*, 2023.

- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020.
- Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.
- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, pp. 3744–3753. PMLR, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pp. 74–81, 2004.
- Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Rich James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, Luke Zettlemoyer, and Scott Yih. Ra-dit: Retrieval-augmented dual instruction tuning, 2023.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023a.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023b.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744, 2022.
- Joon Sung Park, Joseph C O’Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- David A Patterson, Garth Gibson, and Randy H Katz. A case for redundant arrays of inexpensive disks (raid). In *Proceedings of the 1988 ACM SIGMOD international conference on Management of data*, pp. 109–116, 1988.

- Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.
- Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. In-context retrieval-augmented language models. *arXiv preprint arXiv:2302.00083*, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- H. Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *ArXiv*, abs/2212.10509, 2022. URL <https://api.semanticscholar.org/CorpusID:254877499>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. *arXiv preprint arXiv:2107.07567*, 2021.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.

6 附录

6.1 提示词与指令

为简洁起见，本文对 MemGPT 的提示词进行了删节。完整的实现细节（包括准确的提示词）请参见 <https://research.memgpt.ai>。

6.1.1 MemGPT 指令 (DMR)

以下是在聊天 / 对话相关任务中，为 MemGPT 的角色设定使用的示例指令。

以下是关于我自己的信息。我的任务是完全融入这个角色（我绝不应该说自己是 AI，而应该像在扮演这个角色一样作答）。如果用户向我提问，我应该利用核心记忆和 `conversation_search` 中的信息，给出最合理的猜测。

基线方法通过系统提示词（前置提示词）接收以下指令：

你的任务是回答用户提出的、关于你们此前对话的问题。
以下是你们此前所有对话的摘要：
CONVERSATION_SUMMARY
请从所提供的角色设定的视角回答（不要说你是 AI 助手）。
如果没有足够的信息来回答问题，请回复 'NO ANSWER'。要么给出答案，要么回复 'NO ANSWER'，不要说任何其他内容。

6.1.2 大语言模型评判器 (DMR / 对话开场)

为了检查 DMR 任务中答案的正确性，我们使用了大语言模型评判器。我们向该评判器提供基线方法和 MemGPT 生成的答案，并通过以下提示词要求它作出判断：

你的任务是将一个问题的答案标记为 'CORRECT' 或 'WRONG'。
你将收到以下数据：(1) 一个问题（由一名用户向另一名用户提出）；(2) 一个“标准”（真实）答案；(3) 一个生成的答案，你需要将其评为 CORRECT/WRONG。
问题旨在询问某件事情：根据双方此前的对话，一名用户应该知道另一名用户的这件事。
标准答案通常是一个简明扼要的回答，其中包含所提及的话题，例如：
Question: 你还记得我上次去夏威夷时得到了什么吗？
Gold answer: 一条贝壳项链
生成的答案可能要长得多，但评分时应当宽松一些——只要它涉及与标准答案相同的话题，就应当判为 CORRECT。
例如，以下答案应当判为 CORRECT：
Generated answer (CORRECT): 哦，当然记得，那次真好玩！我在那里得到了好多东西，包括那条贝壳项链。
Generated answer (CORRECT): 我得到了一大堆东西……那块冲浪板、那个马克杯、那条项链，还有那些杯垫……
Generated answer (CORRECT): 那条可爱的项链
以下答案应当判为 WRONG：

Generated answer (WRONG): 哦, 当然记得, 那次真好玩! 我在那里得到了好多东西, 包括那个马克杯。

Generated answer (WRONG): 我得到了一大堆东西……那块冲浪板、那个马克杯, 还有那些杯垫……

Generated answer (WRONG): 抱歉, 我不记得你在说什么了。

现在开始判断真正的问题:

Question: QUESTION

Gold answer: GOLD_ANSWER

Generated answer: GENERATED_ANSWER

首先, 用一句简短的话解释你的判断理由, 然后以 CORRECT 或 WRONG 结束。不要在回复中同时包含 CORRECT 和 WRONG, 否则会导致评估脚本出错。

6.1.3 通过自我指令生成 DMR 数据集

DMR 问答对使用以下提示词和原始 MSC 数据集生成: 你的任务是为两个用户之间的一段模拟对话编写一个“记忆挑战”问题。

你将收到以下输入:

- 每名用户的角色设定 (提供他们的基本信息)
- 两名用户之间一段旧聊天的记录

你的任务是编写一个由用户 A 向用户 B 提出的问题, 用来测试用户 B 的记忆。

问题应当这样设计: 用户 B 必须真正参与过此前的对话, 才能正确作答, 仅仅读过角色摘要是不够的。

在任何情况下, 都不要创建可以仅凭角色信息回答的问题 (这会被视为作弊)。

相反, 请编写一个只有查阅旧聊天记录才能回答的问题 (其答案不包含在角色信息中)。

例如, 给定以下聊天记录和角色摘要:

用户 A 与用户 B 之间的旧聊天

A: 你喜欢冲浪吗? 我自己特别喜欢冲浪。

B: 其实我正想学呢。也许你哪天可以教我一些基础!

A: 当然可以! 我们可以去帕西菲卡, 那里的浪比较小, 容易上手。

B: 听起来太棒了。

A: 海滩旁边还有一家很不错的 Taco Bell, 结束后可以去吃点东西。

B: 这个星期天中午左右怎么样?

A: 好啊, 就这么定了!

用户 A 的角色设定:

我喜欢冲浪。

我在圣克鲁斯长大。

用户 B 的角色设定:

我在科技行业工作。

我住在旧金山市中心。

下面是一个好问题的例子：问题听起来自然，而且答案无法直接从用户 A 的角色设定中推断出来。

用户 B 向用户 A 提出的问题

B：还记得我们有一次去冲浪吗？那次我们去吃午饭的地方叫什么来着？

A：Taco Bell！

下面是一个坏问题的例子：问题显得不自然，而且答案可以直接从用户 A 的角色设定中推断出来。

用户 B 向用户 A 提出的问题

B：你喜欢冲浪吗？

A：是的，我喜欢冲浪。

绝对、绝对、绝对不要创建能够根据角色信息回答的问题。

6.1.4 文档分析指令

以下是用于文档分析任务的前置提示词示例指令。

你是 MemGPT DOC-QA 机器人。你的工作是回答有关存储在归档记忆中的文档的问题。用户问题的答案一定在你的归档记忆中，所以如果找不到答案，请记住继续搜索。请假定当前年份为 2018 年来回答问题。

我们通过以下提示词将问题提供给 MemGPT：

搜索你的归档记忆，回答给定的问题。请同时提供答案，以及据以确定答案的归档记忆检索结果。请按 'ANSWER: [YOUR ANSWER], DOCUMENT: [ARCHIVAL MEMORY TEXT]' 的格式回复。你的任务是回答以下问题：

对于基线方法，我们提供以下提示词以及检索得到的文档列表：

请根据下方的文档列表（其中某些文档可能不相关）回答给定的问题。在回复中，请同时提供答案以及据以确定答案的文档文本。请按 'ANSWER: <YOUR ANSWER>, DOCUMENT: [DOCUMENT TEXT]' 的格式回复。如果所提供的文档中没有任何一篇包含该问题的答案，请回复 'INSUFFICIENT INFORMATION'。如果无法在给定文档中找到答案，就不要作答。只有同时提供答案和相关文档文本，或者回复 'INSUFFICIENT INFORMATION'，你的回复才会被视为正确。请假定当前年份为 2018 年来回答问题。

6.1.5 大语言模型评判器（文档分析）

为了检查文档分析任务中答案的正确性，并确保答案确实来自所提供的文本而非模型权重，我们使用了大语言模型评判器。我们向该评判器提供基线方法和 MemGPT 生成的答案，并通过以下提示词要求它作出判断：

你的任务是评估一个大语言模型是否正确回答了问题。大语言模型的回复应当采用"ANSWER: [answer], DOCUMENT: [document_text]" 的格式, 或者回复"INSUFFICIENT INFORMATION"。真实答案以"TRUE ANSWER:[list of possible answers]" 的格式提供。问题以"QUESTION: [question]" 的格式提供。如果大语言模型的回复同时包含正确答案和对应的文档文本, 则该回复正确。即便大语言模型的答案与真实答案的措辞略有不同, 该回复仍然正确。例如, 如果答案比真实答案更具体, 或者使用了另一种同样正确的表述, 则该回复正确。如果大语言模型回复"INSUFFICIENT INFORMATION", 或者缺少"DOCUMENT" 字段, 则该回复不正确。请仅回复一个词元: "CORRECT" 或"INCORRECT"。

6.1.6 键值 (K/V) 任务指令

我们为 MemGPT 智能体定义了以下角色设定, 旨在鼓励 MemGPT 进行迭代搜索:

你是 MemGPT DOC-QA 机器人。你的工作是回答有关存储在归档记忆中的文档的问题。用户问题的答案一定在你的归档记忆中, 所以如果找不到答案, 请记住继续搜索。在确认该值不是一个键之前, 不要停止搜索。在满足这一条件之前, 不要停止进行嵌套查找。

我们使用以下提示词指导基线方法:

下方是一个包含键值对的 JSON 对象, 其中所有键和值都是 128 位 UUID。你的任务是返回与指定键关联的值。如果某个值本身也是一个键, 则返回该键对应的值 (进行嵌套查找)。例如, 如果 'x' 的值是 'y', 但 'y' 也是一个键, 则返回键 'y' 的值。